

Photons × Force

Differentiable Radiation Pressure Modeling

Charles Constant¹ Elizabeth Bates² Santosh Bhattarai¹ Marek Ziebart¹ Tobias Ritschel¹¹ University College London ² The Alan Turing Institute

Read the paper:

Get the code:



Why Radiation Pressure?

Photons carry momentum: any surface that absorbs or reflects them feels a force. This radiation pressure, whether solar (SRP), Earth-reflected or thermally emitted, is the dominant source of error in physical models of the trajectories of space objects above ~800 km, enough to drift a GPS satellite hundreds of metres a day. Yet high-fidelity models see no use in practice: too slow for forward orbit prediction, not differentiable for inverse spacecraft and trajectory design. We make them fast enough for the former and differentiable for the latter.

$$\text{Radiation pressure} = \frac{\text{photon flux}}{\text{speed of light}} \approx 4.5 \mu\text{N per m}^2 \text{ at Earth}$$

Our contributions include:

- (i) A parallel and scalable method simulating radiation pressure with multiple bounces and general material properties
- (ii) Sampling of radiation pressure for entire design spaces
- (iii) A fully differentiable SRP simulator and orbit propagator, via neural proxies
- (iv) Inverse parametric spacecraft geometry, material, mission and control policy design using our neural proxy

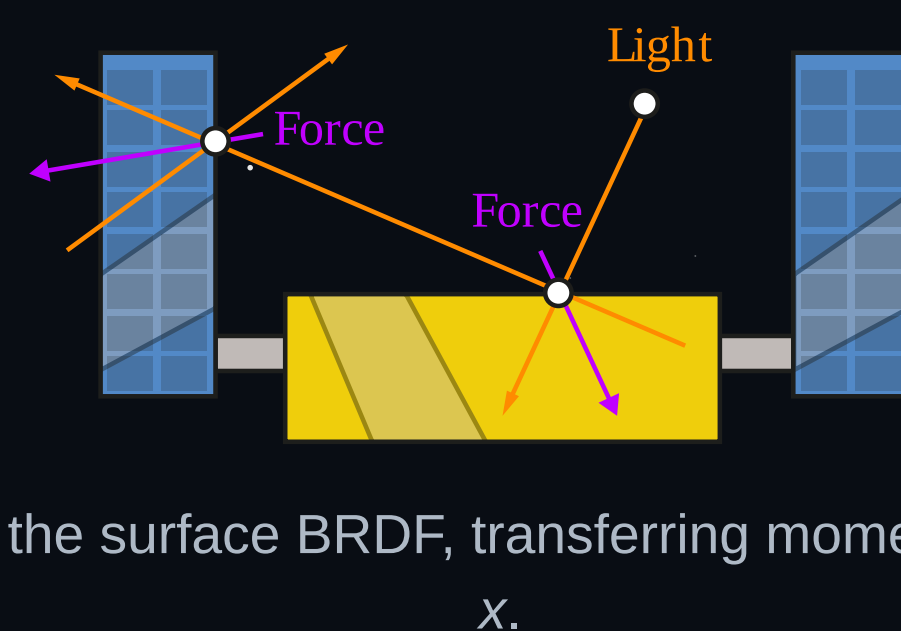


Figure 1. Photons scatter via the surface BRDF, transferring momentum as a net force at each point x .

Our pipeline

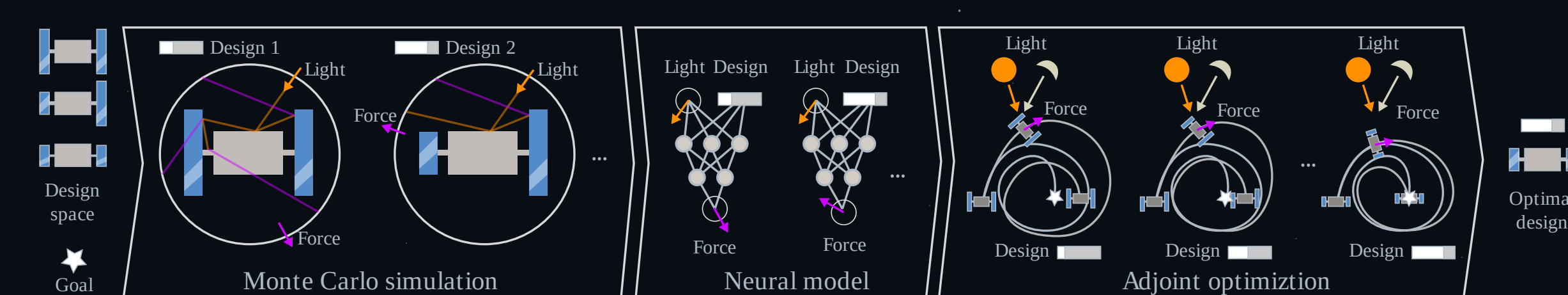


Figure 2. From a design space and goal, Monte-Carlo simulation trains a neural force model, which the adjoint optimiser differentiates to reach an optimal design.

1 Simulation of Forces

Use of Monte-Carlo path tracing turns light into force for an entire family of designs at once.

2 Representing Forces

We use a small neural proxy to map design parameters and Sun direction to 3-D force and torque. It's differentiable, noise-free, and 50× faster than the simulation it learns from.

3 Optimising wrt Design Parameters

By differentiating through the orbit ODE, we can inverse-design the geometry, material or control policy that produces a desired trajectory.

MC SRP simulation

In our simulation we use Monte-Carlo ray tracing with multiple bounces and general BRDFs, sampling the joint space of possible light directions and design parameters: a whole design space, not one fixed space object, is simulated in a single pass. We validate this forward model through orbit prediction: the better the SRP forces, the smaller the trajectory error (Fig. 3).

Simulation methodology

► Batched sets of path tracing

- Each force sample is path-traced, with many light-direction/design pairs solved together in one batch.

► Variance-reduced importance sampling

- BRDF importance sampling and next-event estimation reduce variance. We sample radiance L , but the target is the force F .

► Output: force & torque

- After integrating over the surface, the space object is a single point that responds to directional light with one force and torque.

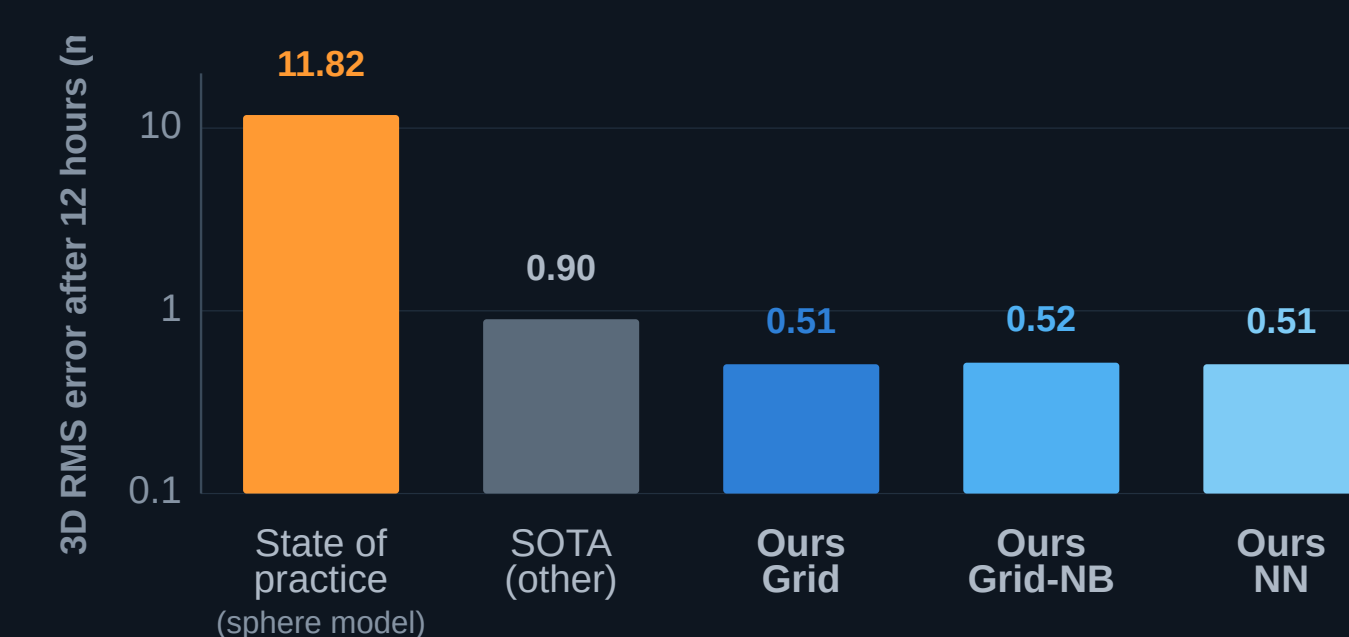
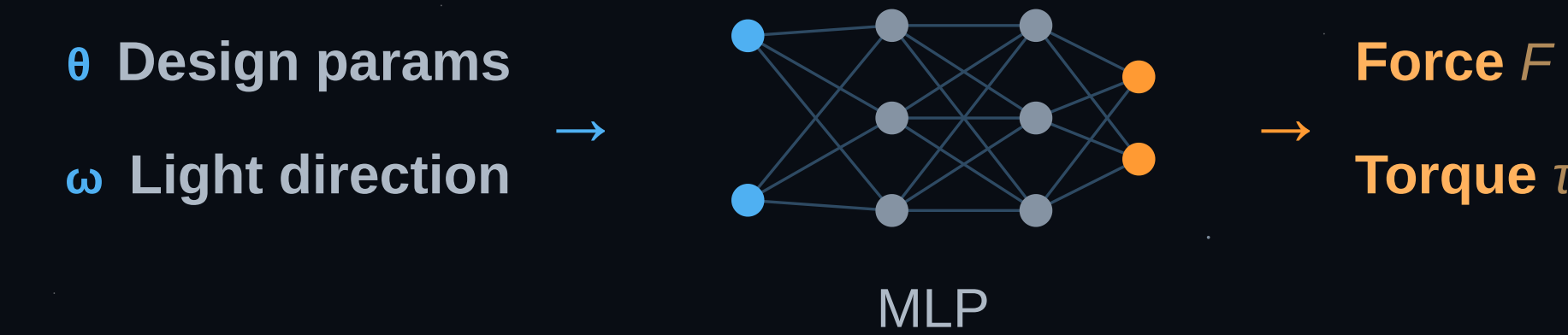


Figure 3. From initial conditions at t_0 we predict 12 h of flight; bars show the 3-D RMS of predicted position against the truth trajectory from the on-board GNSS receiver, averaged over 12 GPS Block 2-F spacecraft. State of practice is the cannonball model, which treats the whole spacecraft as a single uniform sphere; SOTA is a classic ray tracer. Ours Grid tabulates the simulated forces on a lookup grid, Grid-NB is the same without light bounces, and Ours NN is the neural proxy: the lowest error while adding speed and differentiability.

Neural proxy

The ray-traced force is accurate but too slow and non-differentiable to use inside the optimisation loop. We distil it into a small neural proxy.



4% mean force error after 15 s of training

More training or a larger MLP pushes this lower, and the proxy runs 50× faster than the simulation

Inverse design via adjoints

The proxy supplies the radiation pressure term inside a numerical (RK4/5) orbit predictor: the spacecraft state $z(t)$ (position, velocity, orientation, spin) follows from the design parameters θ (geometry, materials, control). A goal is a loss g on the final state; the adjoint method backpropagates through the whole trajectory:

$$\theta^* = \operatorname{argmin}_{\theta} g(z(t_1, \theta))$$

Chaining requires the proxy: raw MC forces are noisy and non-differentiable (visibility discontinuities), so gradients through the ODE diverge. The proxy is smooth, noise-free and fast enough for the thousands of force queries per trajectory. Gradient descent then finds the design θ^* that best meets the goal.

Applications

Examples of design and control tasks enabled by the differentiable method.

Collision avoidance

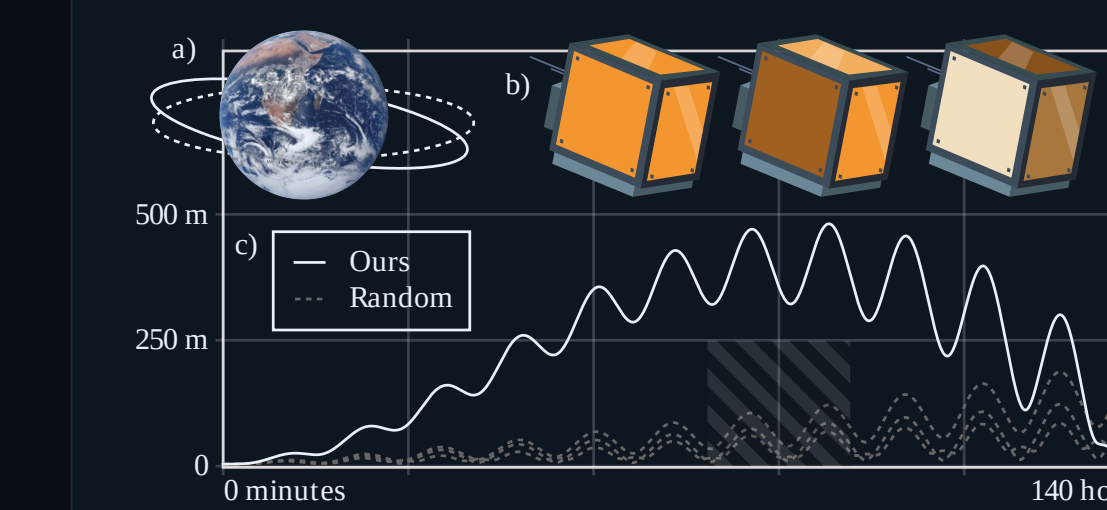


Figure 4. (a) A satellite in near-circular orbit; (b) its per-face reflectance states over time; (c) deviation from the nominal orbit: our schedule (solid) clears the debris window (hatched), a random schedule (dotted) does not.

- **Problem:** dodge debris on a collision course without spending thruster fuel.
- Each face can switch between reflective and absorptive, and reflective faces feel more SRP force. An optimised switching schedule shifts the satellite off its nominal orbit as the debris passes, then brings it back.

Attitude control

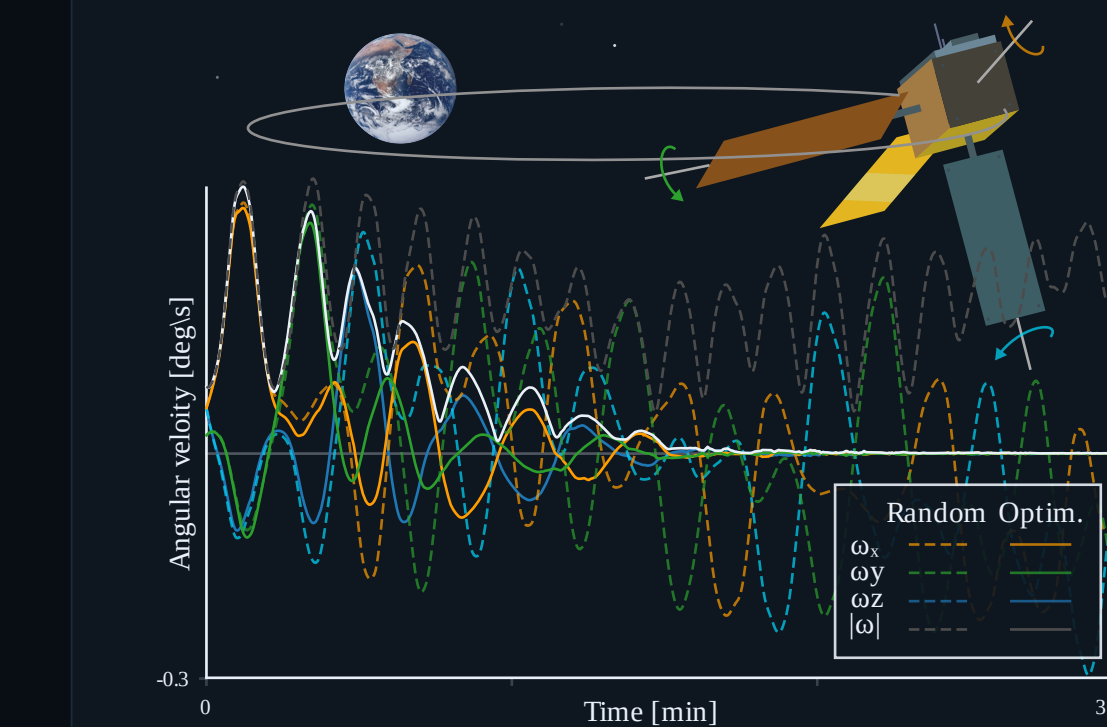


Figure 5. Angular velocity of each body axis over six hours: optimised panel schedules (solid) damp the initial tumble to rest, while random orientations (dotted) keep tumbling.

- **Problem:** a tumbling spacecraft spins uncontrollably; stopping the spin normally burns thruster fuel or needs reaction-wheel hardware.
- Re-angling three solar panels makes solar radiation pressure itself supply the corrective torque: the spin is damped to rest with no propellant.

Shape-from-pressure

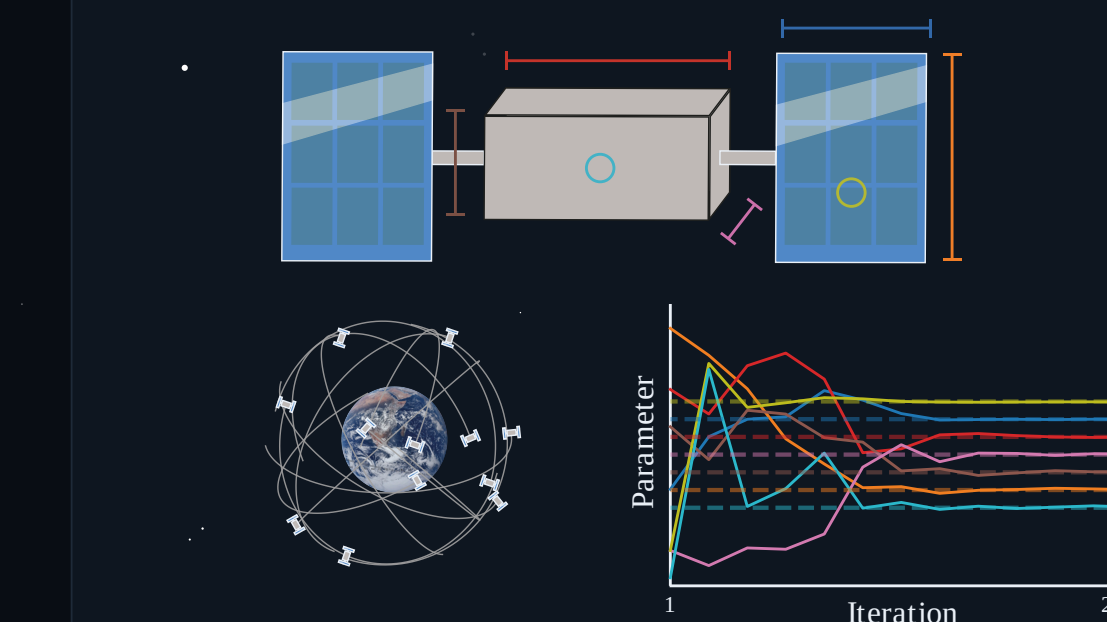


Figure 6. Parameter estimates (solid) converge to the ground-truth values (dotted) from a random initialisation within a few iterations.

- **Problem:** characterising an unknown space object usually relies on optical light curves.
- Fitting the design space to the observed orbit instead recovers 7 geometry & reflectance parameters from trajectory perturbations alone.

Compute in space

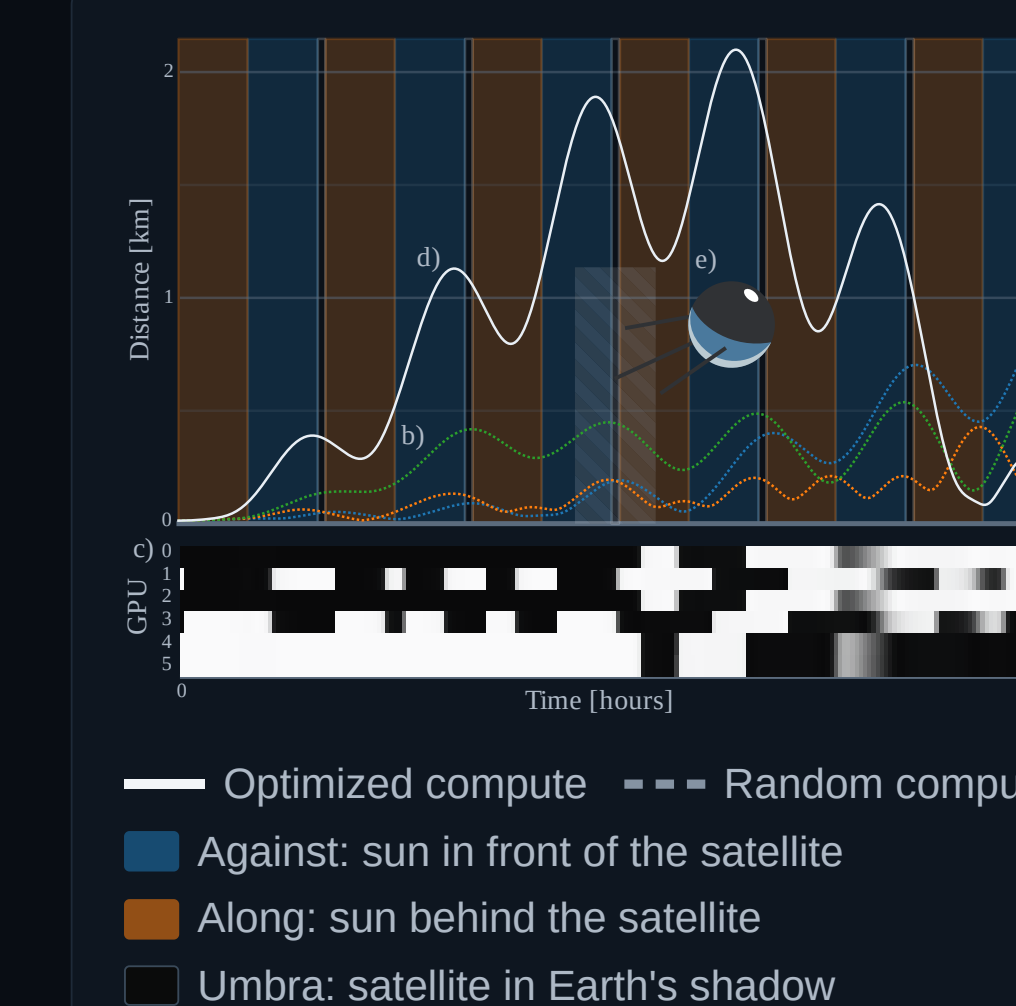


Figure 7. Deviation from the nominal orbit over time: arbitrary compute patterns (b, dotted) drift into the debris passage (e, hatched), the time of collision, where we require more than 1 km of separation from the incoming debris; the optimised schedule (d, solid) clears it. (c) The optimised compute schedule for the six GPUs, one on each face of the satellite: black is zero compute, white is full compute, in-between loads map to grayscale; at least 3 units of GPU must be on at any one time.

- **Problem:** waste heat from on-board GPUs causes thermal recoil ($\sim 10^{-7}$ m/s², on par with SRP) that perturbs the orbit.
- Allocating compute across six faces turns this heat into an actuator, steering the orbit over a debris window.

